恒生电子股份有限公司

Java 编码规范

说　明

本文档中所包含的信息属于商业机密信息，如无恒生电子股份有限公司的书面许可，任何人都无权复制或利用。

目 录

一、概述.....3
二、术语及专用名词解释.....3
三、约束等级定义.....3
四、编码规约.....5
1 命名风格.....5
2 常量定义.....8
3 代码格式.....11
4 OOP 规约.....14
5 集合处理.....22
6 并发处理.....26
7 控制语句.....29
8 注释规约.....34
9 异常处理.....36
10 日志规约.....39
11 二方库依赖.....41
12 其它.....43
五、IDE 配置项设置.....45
1 文件注释.....45
2 编码格式.....47
3 缩进设置.....47
六、相关规定.....48
七、修订记录.....48

一、 概述

本规范旨在为全公司 java 程序的开发者提供规范化最新的指导，可用于程序员个人编译环境以及研发团队集成环境等场合的代码规范化检查。

二、 术语及专用名词解释

- POJO（ Plain Ordinary Java Object ）：在本文档中，POJO 专指只有 setter / getter / toString 的简单类，包括 DO/DTO/BO/VO 等。
- GAV（ GroupId、ArtifactctId、Version ）：Maven 坐标，是用来唯一标识 jar 包。
- OOP（ Object Oriented Programming ）：本文档泛指类、对象的编程处理方式。
- ORM（ Object Relation Mapping ）：对象关系映射，对象领域模型与底层数据之间的转换，本文泛指 hibernate，mybatis 等框架。
- NPE（ java.lang.NullPointerException ）：空指针异常。
- SOA（ Service-Oriented Architecture ）：面向服务架构，它可以根据需求通过网络对松散耦合的粗粒度应用组件进行分布式部署、组合和使用，有利于提升组件可重用性，可维护性。
- IDE（ Integrated Development Environment ）：用于提供程序开发环境的应用程序，一般包括代码编辑器、编译器、调试器和图形用户界面等工具，本文档泛指 IntelliJ IDEA 和 eclipse。
- OOM（ Out Of Memory ）：源于 java.lang.OutOfMemoryError，当 JVM 没有足够的内存来为对象分配空间并且垃圾回收器也无法回收空间时，系统出现的严重状况。
- 一方库：本工程内部子项目模块依赖的库（jar 包）。
- 二方库：公司内部发布到中央仓库，可供公司内部其它应用依赖的库（jar 包）。
- 三方库：公司之外的开源库（jar 包）。
- 插件扫描：指该要求条目的检查可以用 IDE 工具上或在集成环境中加载插件的方式进行。

三、 约束等级定义

约束等级	约束效力	强制性
【强制】	违反该项将被认为代码存在严重缺陷	全公司所有 Java 编码团队必须遵守

【推荐】	违反该项即被认为代码存在轻微缺陷	各 JAVA 编码团队根据具体产品特性的不同，选择性地遵守
【参考】	违反该项可被认为代码存在优化空间	各 JAVA 编码团队从产品持续优化及人员技能提升的角度，参考使用

四、 编码规约

1 命名风格

1.1 【强制】代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。

检查方式：插件扫描

反例： _name / __name / $name / name_ / name$ / name__

1.2 【强制】代码中的命名严禁使用拼音与英文混合的方式，更不允许直接使用中文的方式。

说明：正确的英文拼写和语法可以让阅读者易于理解，避免歧义。注意，即使纯拼音命名方式也要避免采用。

检查方式：人工检查

正例：hundsun / hangzhou 等国际通用的名称，可视同英文。

反例：DaZhePromotion [打折] / getPingfenByName() [评分] / int 某变量 = 3

1.3 【强制】类名使用 UpperCamelCase 风格，但以下情形例外： DTO / UID/DO/VO/DAO/BO/DAOImpl/YunOS/AP/PO 等。

检查方式：插件扫描

正例：MarcoPolo / UserDTO / XmlService / TcpUdpDeal / TaPromotion

反例：macroPolo / UserDto / XMLService / TCPUDPDeal / TAPromotion

1.4 【强制】方法名、参数名、成员变量、局部变量都统一使用 lowerCamelCase 风格，必须遵从驼峰形式。

检查方式：插件扫描

正例：localValue / getHttpMessage() / inputUserId

1.5 【强制】常量命名全部大写，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长。

检查方式：插件扫描

正例：MAX _ STOCK _ COUNT

反例：MAX _ COUNT

1.6 【推荐】抽象类命名使用 Abstract 开头 ； 异常类命名使用 Exception 结尾 ； 测试类命名以它要测试的类的名称开始，以 Test 结尾。

检查方式：插件扫描

1.7【强制】类型与中括号紧挨相连来表示数组。

检查方式：插件扫描

正例：定义整形数组 int[] arrayDemo;

反例：在 main 参数中，使用 String args[]来定义。

1.8【强制】 POJO 类中布尔类型的变量，都不要加 is 前缀 ，否则部分框架解析会引起序列化错误。

检查方式：插件扫描

反例：定义为基本数据类型 Boolean isDeleted 的属性，它的方法也是 isDeleted() ， RPC 框架在反向解析的时候，“误以为”对应的属性名称是 deleted ，导致属性获取不到，进而抛出异常。

1.9【强制】包名统一使用小写，点分隔符之间有且仅有一个自然语义的英语单词。包名统一使用单数形式，但是类名如果有复数含义，类名可以使用复数形式。

检查方式：插件扫描

正例：应用工具类包名为 com.hundsun.jres.util 、类名为 MessageUtils（此规则参考 spring 的框架结构 ）

1.10 【强制】杜绝完全不规范的缩写，避免望文不知义。

检查方式：人工检查

反例：AbstractClass “缩写”命名成 AbsClass；condition “缩写”命名成 condi ，此类随意缩写严重降低了代码的可阅读性。

1.11 【推荐】为了达到代码自解释的目标，任何自定义编程元素在命名时，使用尽量完整的单词组合来表达其意。

检查方式：人工检查

正例：在 JDK 中，表达原子更新的类名为：AtomicReferenceFieldUpdater。

反例：变量 int a 的随意命名方式。

1.12 【推荐】如果模块、接口、类、方法使用了设计模式，在命名时需体现出具体模式。

检查方式：人工检查

说明：将设计模式体现在名字中，有利于阅读者快速理解架构设计理念。

正例： public class OrderFactory;

public class LoginProxy;

public class ResourceObserver;

1.13 【推荐】接口类中的方法和属性不要加任何修饰符号（public 也不要加），保持代码的简洁性，并加上有效的 Javadoc 注释。尽量不要在接口里定义变量，如果一定要定义变量，肯定是与接口方法相关，并且是整个应用的基础常量。

检查方式：插件扫描

正例：接口方法签名 void commit();

接口基础常量 String COMPANY = " hundsun " ;

反例：接口方法定义 public abstract void f();

说明： JDK 8 中接口允许有默认实现，那么这个 default 方法，是对所有实现类都有价值的默认实现。

1.14 接口和实现类的命名有两套规则：

- 【强制】对于 Service 和 DAO 类，基于 SOA 的理念，暴露出来的服务一定是接口，内部的实现类用 Impl 的后缀与接口区别。

检查方式：插件扫描

正例：CacheServiceImpl 实现 CacheService 接口。

说明： 遗留系统确实有很多以 I 开头的接口定义，例如 ICacheService 接口，CacheService 为实现类，对于遗留系统可以不做强制要求，对于新研发的项目需要做强制要求

- 【推荐】 如果是形容能力的接口名称，取对应的形容词为接口名（通常是 - able 的形式）。

检查方式：人工检查

正例：AbstractTranslator 实现 Translatable 接口 。

1.15 【参考】枚举类名建议带上 Enum 后缀，枚举成员名称需要全大写，单词间用下划线隔开。

检查方式：插件扫描

说明：枚举其实就是特殊的类，域成员均为常量，且构造方法被默认强制是私有。

正例：枚举名字为 ProcessStatusEnum 的成员名称：SUCCESS / UNKNOWN _ REASON。

1.16 【推荐】代码中浮点数建议使用 BigDecimal 类型，谨慎使用 float、double，根据业务需要定义好精度（计算、入表）。float 和 double 只能用来做科学计算或者是工程计算，在商业计算中我们

要用 java.math.BigDecimal。

检查方式：插件扫描

说明： float 和 double 类型执行二进制浮点运算，这是为了在广泛的数值范围上提供较为精确的快速计算而精心设计的，然而，他们并没有提供完全精确的结果，所以不应该被用于精确的结果的场合。

1.17 【强制】BigDecimal 禁止使用 double 构造函数

检查方式：插件检查

1.18 【推荐】静态工具方法的类可以定义为抽象类型 Abstract

检查方式：人工检查

说明：防止工具类被实例化

1.19 【参考】各层命名规约：

- Service 层方法命名规约，服务接口的名称按照“动作+资源(表)+业务标识”进行标识，采用驼峰风格，其中动作包括 get、put、post、delete 对应查询、修改、增加和删除；

检查方式：插件扫描

- 获取单个对象的方法用 get 做前缀。
- 获取多个对象的方法用 get 做前缀，复数形式结尾如：getObjects。
- 插入的方法用 post 做前缀。
- 删除的方法用 delete 做前缀。
- 修改的方法用 put 做前缀。

- DAO (Data Access Object) 数据访问对象是一个面向对象的数据库接口，DAO 层方法命名规约

检查方式：人工检查

- 获取单个对象的方法用 get 做前缀。
- 获取多个对象的方法用 list 做前缀，复数形式结尾如：listObjects。
- 获取统计值的方法用 count 做前缀。
- 插入的方法用 insert 做前缀。
- 删除的方法用 delete 做前缀。
- 修改的方法用 update 做前缀。

2 常量定义

2.1 【强制】条件判断语句里不允许任何魔法值 （ 即未经预先定义的常量 ） 直接出现在代码中。

检查方式：插件扫描

反例：

```
if ("client_id".equals(checkKey)) {
    if (output == null) {
        return output;
    }
    output.setClientId(dataAuthorityController.parseData(checkKey,
    output.getClientId()));
}
```

2.2【强制】在 long 或者 Long 赋值时，数值后使用大写的 L ，不能是小写的英文字母 l ，因为它容易跟数字 1 混淆，造成误解。

检查方式：插件扫描

2.3【推荐】不要使用一个常量类维护所有常量，要按常量功能进行归类，分开维护。

检查方式：人工检查

说明：大而全的常量类，杂乱无章，使用查找功能才能定位到修改的常量，不利于理解和维护。

正例：缓存相关常量放在类 CacheConsts 下 ；系统配置相关常量放在类 ConfigConsts 下。

2.4【推荐】常量的复用层次有五层：跨应用共享常量、应用内共享常量、子工程内共享常量、包内共享常量、类内共享常量。

检查方式：人工检查

- 跨应用共享常量：放置在二方库中，通常是 client . jar 中的 constant 目录下。
- 应用内共享常量：放置在一方库中，通常是子模块中的 constant 目录下。

反例：易懂变量也要统一定义成应用内共享常量，两位攻城师在两个类中分别定义了表示“是”的变量：

类 A 中： public static final String YES = ″ yes ″ ;

类 B 中： public static final String YES = ″ y ″ ;

A . YES . equals(B . YES) ，预期是 true ，但实际返回为 false ，导致线上问题。

- 子工程内部共享常量：即在当前子工程的 constant 目录下。

- 包内共享常量：即在当前包下单独的 constant 目录下。
- 类内共享常量：直接在类内部 private static final 定义。

2.5 【推荐】如果变量值仅在一个固定范围内变化用 enum 类型来定义。

检查方式：人工检查

说明：如果存在名称之外的延伸属性应使用 enum 类型，下面正例中的数字就是延伸信息，表示一年中的第几个季节。

正例：

```
public enum SeasonEnum {
    SPRING(1), SUMMER(2), AUTUMN(3), WINTER(4);
    private int seq;
    SeasonEnum(int seq){
        this.seq = seq;
    }
}
```

3 代码格式

3.1 【强制】大括号的使用约定。如果是大括号内为空，则简洁地写成{}即可，不需要换行；如果是非空代码块则：

检查方式：代码扫描

- 左大括号前不换行。
- 左大括号后换行。
- 右大括号前换行。
- 右大括号后还有 else 等代码则不换行 ； 表示终止的右大括号后必须换行。

3.2 【推荐】左小括号和右边的第一个字符之间不出现空格 ； 同样，右小括号和左边的第一个字符之间也不出现空格；而左大括号前需要空格。详见第 5 条下方正例提示。

检查方式：人工检查

反例：if (空格 a == b 空格)

3.3 【推荐】 if / for / while / switch / do 等保留字与括号之间都必须加空格。

检查方式：人工检查

3.4 【推荐】任何二目、三目运算符的左右两边都需要加一个空格。

检查方式：人工检查

说明：运算符包括赋值运算符=、逻辑运算符&&、加减乘除符号等。

3.5 【强制】采用 4 个空格缩进，禁止使用 tab 字符。

检查方式：插件扫描

说明：如果使用 tab 缩进，必须设置 1 个 tab 为 4 个空格。IDEA 设置 tab 为 4 个空格时，请勿勾选 Use tab character ；而在 eclipse 中，必须勾选 insert spaces for tabs 。

正例：（ 涉及 1-5 点 ）

```
    public static void main(String[] args) {
        // 缩进 4 个空格
        String say = "hello";
        // 运算符的左右必须有一个空格
        int flag = 0;
        // 关键词 if 与括号之间必须有一个空格，括号内的 f 与左括号，0 与右括号不需要空格
        if (flag == 0) {
```

```
            System.out.println(say);
        }
        // 左大括号前加空格且不换行；左大括号后换行
        if (flag == 1) {
            System.out.println("world");
        // 右大括号前换行，右大括号后有 else，不用换行
        } else {
            System.out.println("ok");
        // 在右大括号后直接结束，则必须换行
        }
    }
```

3.6【推荐】注释的双斜线与注释内容之间有且仅有一个空格。

检查方式：人工检查

正例：

```
// 这是示例注释，请注意在双斜线之后有一个空格
String ygb = new String();
```

3.7【推荐】单行字符数限制不超过 120 个，超出需要换行，换行时遵循如下原则：

- 第二行相对第一行缩进 4 个空格，从第三行开始，不再继续缩进，参考示例。
- 运算符与下文一起换行。
- 方法调用的点符号与下文一起换行。
- 方法调用中的多个参数需要换行时，在逗号后进行。
- 在括号前不要换行，见反例。

检查方式：插件扫描

正例：

```
    StringBuffer sb = new StringBuffer();
    // 超过 120 个字符的情况下，换行缩进 4 个空格，点号和方法名称一起换行
    sb.append("zi").append("xin")...
        .append("huang")...
       .append("huang")...
       .append("huang");
```

反例：

```
StringBuffer sb = new StringBuffer();
// 超过 120 个字符的情况下，不要在括号前换行
sb.append("zi").append("xin")...append
      ("huang");
// 参数很多的方法调用可能超过 120 个字符，不要在逗号前换行
method(args1, args2, args3, ...
      , argsX);
```

3.8 【强制】方法参数在定义和传入时，多个参数逗号后边必须加空格。

检查方式：插件扫描

正例：下例中实参的 args1，后边必须要有一个空格。

```
method(args1, args2, args3);
```

3.9 【强制】 IDE 的 text file encoding 设置为 UTF -8 ; IDE 中文件的换行符使用 Unix 格式，不要使用 Windows 格式。

3.10 检查方式：插件扫描【推荐】单个方法的总行数不超过 120 行。

检查方式：插件扫描

说明：包括方法签名、结束右大括号、方法内代码、注释、空行、回车及任何不可见字符的总行数不超过 120 行。

正例：代码逻辑分清红花和绿叶，个性和共性，绿叶逻辑单独出来成为额外方法，使主干代码更加清晰；共性逻辑抽取成为共性方法，便于复用和维护。

3.11 【推荐】没有必要增加若干空格来使某一行的字符与上一行对应位置的字符对齐。

检查方式：人工检查

正例：

```
int one = 1;
long two = 2L;
float three = 3F;
StringBuffer sb = new StringBuffer();
```

说明：增加 sb 这个变量，如果需要对齐，则给 a 、b 、c 都要增加几个空格，在变量比较多的情况下，是非常累赘的事情。

3.12 【推荐】不同逻辑、不同语义、不同业务的代码之间插入一个空行分隔开来以提升可读性。

检查方式：人工检查

说明：任何情形，没有必要插入多个空行进行隔开。

3.13 【强制】一行内不能有多条语句。

检查方式：插件扫描

反例 1：

```
 int var1 = 1
 ; var2 = 2; //violation here
int o = 1, p = 2,
r = 5; int t; //violation here
```

反例 2：

```
try (Reader r = new PipedReader(); s2; Reader s3 =
new PipedReader() // violation
) {}
```

说明：提高代码的易维护性

3.14 【强制】Dot（.）前不允许有空格，可以换行。

检查方式：插件扫描

正例 1：

```
sb.append("you").append("are").append("lucky");
```

正例 2：

```
sb.append("zi").append("xin")
.append("huang")
.append("huang")
.append("huang");
```

说明：Java 规范标准格式

4 OOP 规约

4.1 【强制】避免通过一个类的对象引用访问此类的静态变量或静态方法，无谓增加编译器解析成本，直接用类名来访问即可。

检查方式：插件扫描

4.2 【强制】所有的覆写方法，必须加@ Override 注解。

检查方式：插件扫描

说明：getObject() 与 getObject() 的问题。一个是字母的 O ，一个是数字的 0，加@ Override 可以准确判断是否覆盖成功。另外，如果在抽象类中对方法签名进行修改，其实现类会马上编译报错。

4.3 【强制】相同参数类型，相同业务含义，才可以使用 Java 的可变参数，避免使用 Object 。

检查方式：插件扫描

说明：可变参数必须放置在参数列表的最后。（提倡同学们尽量不用可变参数编程）

正例：public List<User> listUsers(String type, Long... ids) {...}

4.4【强制】外部正在调用或者二方库依赖的接口，不允许修改方法签名，避免对接口调用方产生影响。接口过时必须加@ Deprecated 注解，并清晰地说明采用的新接口或者新服务是什么。

检查方式：HEP 系统检查

4.5【强制】不能使用过时的类或方法。

检查方式：插件扫描

说明：java . net . URLDecoder 中的方法 decode(String encodeStr) 这个方法已经过时，应该使用双参数 decode(String source, String encode)。接口提供方既然明确是过时接口，那么有义务同时提供新的接口；作为调用方来说，有义务去考证过时方法的新实现是什么。

4.6【强制】Object 的 equals 方法容易抛空指针异常，应使用常量或确定有值的对象来调用 equals。

检查方式：插件扫描

正例："test".equals(object);

反例：object.equals("test");

说明：推荐使用 java . util . Objects # equals（JDK 7 引入的工具类）

4.7【强制】所有的相同类型的包装类对象之间值的比较，全部使用 equals 方法比较。

检查方式：插件扫描

说明：对于 Integer var = ? 在-128 至 127 范围内的赋值，Integer 对象是在 IntegerCache . cache 产生，会复用已有对象，这个区间内的 Integer 值可以直接使用==进行判断，但是这个区间之外的所有数据，都会在堆上产生，并不会复用已有对象，这是一个大坑，推荐使用 equals 方法进行判断。

4.8【强制】String 类型之间值的比较，必须使用 equals 方法比较，禁止使用==或!=进行比较。

检查方式：插件扫描

4.9 关于基本数据类型与包装数据类型的使用标准如下：

- 【强制】所有的 POJO 类属性必须使用包装数据类型。

- 【强制】 RPC 方法的返回值和参数必须使用包装数据类型。
- 【推荐】所有的局部变量使用基本数据类型。

检查方式：插件扫描

说明：POJO 类属性没有初值是提醒使用者在需要使用时，必须自己显式地进行赋值，任何 NPE 问题，或者入库检查，都由使用者来保证。

正例：数据库的查询结果可能是 null ，因为自动拆箱，用基本数据类型接收有 NPE 风险。

反例：比如显示成交总额涨跌情况，即正负 x %， x 为基本数据类型，调用的 RPC 服务，调用不成功时，返回的是默认值，页面显示为 0%，这是不合理的，应该显示成中划线。所以包装数据类型的 null 值，能够表示额外的信息，如：远程调用失败，异常退出。

4.10 【参考】定义 DO / DTO / VO 等 POJO 类时，不要设定任何属性默认值。

检查方式：人工检查

反例：POJO 类的 gmtCreate 默认值为 new Date()， 但是这个属性在数据提取时并没有置入具体值，在更新其它字段时又附带更新了此字段，导致创建时间被修改成当前时间。

4.11 【强制】序列化类新增属性时，请不要修改 serialVersionUID 字段，避免反序列失败 ；如果完全不兼容升级，避免反序列化混乱，那么请修改 serialVersionUID 值。

检查方式：人工检查

说明：注意 serialVersionUID 不一致会抛出序列化运行时异常。

4.12 【强制】构造方法里面禁止加入任何业务逻辑（只能用来做注入），如果有初始化逻辑，请放在 init 方法中。

检查方式：人工检查

4.13 【强制】 POJO 类必须写 toString 方法。使用 IDE 中的工具：source > generate toString 时，如果继承了另一个 POJO 类，注意在前面加一下 super . toString 。

检查方式：插件扫描

说明：在方法执行抛出异常时，可以直接调用 POJO 的 toString() 方法打印其属性值，便于排查问题。

4.14 【强制】禁止在 POJO 类中，同时存在对应属性 xxx 的 isXxx() 和 getXxx() 方法。

检查方式：插件扫描

说明：框架在调用属性 xxx 的提取方法时，并不能确定哪个方法一定是被优先调用到。

4.15 【推荐】使用索引访问用 String 的 split 方法得到的数组时，需做最后一个分隔符后有无内容的检查，否则会有抛 IndexOutOfBoundsException 的风险。

检查方式：人工检查

说明：

```
String str = "a,b,c,,";
String[] ary = str.split(",");
// 预期大于 3，结果是 3
System.out.println(ary.length);
```

4.16 【推荐】当一个类有多个构造方法，或者多个同名方法，这些方法应该按顺序放置在一起，便于阅读，此条规则优先于第 16 条规则。

检查方式：人工检查

4.17 【推荐】 类内方法定义的顺序依次是：公有方法或保护方法 > 私有方法 > getter / setter 方法。

检查方式：人工检查

说明：公有方法是类的调用者和维护者最关心的方法，首屏展示最好；保护方法虽然只是子类关心，也可能是“模板设计模式”下的核心方法；而私有方法外部一般不需要特别关心，是一个黑盒实现；因为承载的信息价值较低，所有 Service 和 DAO 的 getter / setter 方法放在类体最后。

4.18 【推荐】 setter 方法中，参数名称与类成员变量名称一致， this . 成员名 = 参数名。在 getter / setter 方法中，不要增加业务逻辑，增加排查问题的难度。

检查方式：人工检查

反例：

```
public Integer getData() {
    if (condition) {
        return this.data + 100;
    } else {
        return this.data - 100;
    }
```

```
    }
```

4.19 【推荐】循环体内，字符串的连接方式，使用 StringBuilder 的 append 方法进行扩展。

检查方式：插件扫描

说明：下例中，反编译出的字节码文件显示每次循环都会 new 出一个 StringBuilder 对象，然后进行 append 操作，最后通过 toString 方法返回 String 对象，造成内存资源浪费。

反例：

```
String str = "start";
for (int i = 0; i < 100; i++) {
    str = str + "hello";
}
```

4.20 【推荐】 final 可以声明类、成员变量、方法、以及本地变量，下列情况使用 final 关键字：

检查方式：人工检查

- 不允许被继承的类，如： String 类。
- 不允许修改引用的域对象。
- 不允许被重写的方法，如： POJO 类的 setter 方法。
- 不允许运行过程中重新赋值的局部变量。
- 避免上下文重复使用一个变量，使用 final 描述可以强制重新定义一个变量，方便更好地进行重构。

4.21 【推荐】慎用 Object 的 clone 方法来拷贝对象。

检查方式：人工检查

说明：对象的 clone 方法默认是浅拷贝，若想实现深拷贝需要重写 clone 方法实现域对象的深度遍历式拷贝。

4.22 【推荐】类成员与方法访问控制从严：

- 如果不允许外部直接通过 new 来创建对象，那么构造方法必须是 private 。
- 工具类不允许有 public 或 default 构造方法。
- 类非 static 成员变量并且与子类共享，必须是 protected 。
- 类非 static 成员变量并且仅在本类使用，必须是 private 。
- 类 static 成员变量如果仅在本类使用，必须是 private 。

- 若是 static 成员变量，考虑是否为 final 。
- 类成员方法只供类内部调用，必须是 private 。
- 类成员方法只对继承类公开，那么限制为 protected 。

检查方式：人工检查

说明：任何类、方法、参数、变量，严控访问范围。过于宽泛的访问范围，不利于模块解耦。

4.23 【强制】服务层方法的返回值和参数必须使用包装数据类型。

检查方式：插件扫描

4.24 【推荐】定义 DO/DTO/VO 等 POJO 类时，不要设定任何属性默认值。

POJO 类属性没有初值是提醒使用者在需要使用时，必须自己显式地进行赋值，任何 NPE 问题，或者入库检查，都由使用者来保证。

检查方式：插件扫描

4.25 【强制】禁止重复 import 同一个类。

检查方式：插件扫描

说明：Java 规范

4.26 【强制】禁止 import 没用的包。

检查方式：插件扫描

说明：Java 规范

4.27 【强制】禁止在方法内对引用类型的入参赋值

说明：如果在方法对引用类型的入参进行赋值，将改变内存引用地址，将变量指向另一块内存区域。但由于形参的改变不影响实参，所以方法外的变量不会跟随一起改变。这在开发过程中很容易造成误判。

反例 1：

```
public static void main(String[] args) {
Map<String, String> a = new HashMap<>();
a.put("key", "5");
updateMap(a);
System.out.println(a);
}
private static void updateMap(Map<String, String> var) {
var = new HashMap<>();
var.put("key", "10");
}
```

在上述方法中，编译不会报错，输出依然是 5。写的人可能期望在

updateMap 这个方法中改变 map 的值。但是在方法内部重新赋值后，入参不再指向原来的内存区域，因此后续的 put 操作没有起到实际的作用。

反例 2：

```
public static void resortList(List<String> tmp){
// 进行从小到大的排序
List<String> toSort = new ArrayList<>();
toSort.addAll(tmp);
List<String> sorted = sortStoL(toSort);
tmp = sorted ;
}
```

上述排序方法，编译不会报错，可以正常执行，但实际不会发生有效的排序。原因是没有直接对原入参对象进行排序，而是对拷贝的对象进行了排序，此时把拷贝对象赋值给入参，仅仅是把入参指向了拷贝对象。但方法外的原 List 本身不会发生任何变化。

4.28 【强制】禁止 import 以下类： java.sql.Date, java.sql.Time, java.sql.Timestamp

检查方式：插件扫描

说明： 当我们在 Java 中使用 Date 类型的时候，预期总是认为其中包含了完整的日期时间信息，但 java.sql.Date 其实只包含日期，可是它又有一个 getHours()方法，当调用 java.sql.Date.getHours()时必定抛出异常，但编译却能通过。如果在代码中将 java.sql.Date 赋值给 java.util.Date 也能编译通过，而 java.util.Date 是可以正常使用 getHours()方法的，这就更容易造成误用。 java.sql.Time 也是类似，它在调用方法 getYear()、getMonth()、getDay()、getDate()时，都会直接抛异常。 java.sql.Timestamp 虽然没有直接抛异常的方法，但是首先它的构造方法、valueOf 方法，toString 方法中均做了很多无用的计算，用来生成固定的日期格式以及纳秒数，造成性能浪费。其次，从名称上看很容易认为自己会获得的是一个时间毫秒数，但其实 通过 java.sql.Timestamp 你能获得是一个包含了[年-月-日 时:分:秒 毫秒.纳秒]的字符串，这就很容易造成误用。

如果要获取毫秒数，在 java 中建议使用 System.currentTimeMillis()，如果要获取纳秒数，可以使用 System.nanoTime()。大多数情况下（例如在 POJO 中使用时），可以用 java.util.Date 代替，实际上被禁止的 3 个类都是 java.util.Date 的子类。

4.29 【强制】禁止使用 System.out.* 或者 System.err.*。

检查方式：插件扫描

说明：System.out.println 将对象结果输出到控制台，会花费大量的 CPU 资源

4.30 【强制】禁止代码中出现类型强转编码，如果确实无法避免，必须在转换前增加类型判断。

（即禁止出现： T t = (T) obj ; 代码层无法避免的情况下，需要有 obj instanceof T 判断）

检查方式：插件扫描

说明：编码应该定义自己需要使用到的接口类型，不同类型原则上应该定义不同接口处理；无法避免的情况下，不允许出现不做保护场景下的类型强转。

4.31 【强制】枚举类做为常量的一种实现方式，要求所有枚举成员属性只读，确保枚举实例内部属性不被外部修改。

检查方式：插件扫描

说明：枚举其实就是常量的一种实现方式，常量不应该被修改，所以需要确保枚举类内部方法不会对枚举成员变量属性进行修改，枚举类的成员变量定义必须增加 final 修饰词。

4.32 【推荐】尽量少用@Resource 来注入 Bean。

检查方式：人工检查

说明：@Resource 优先通过名称进行匹配，当两个类型不同的 bean，名称相同时，不会检查注入是否合理，容易产生 BeanNotOfRequiredTypeException

反例：

```
public class A{
        @Resource
        private BeanTypeA a;
}
public class BeanTypeB{
      @Bean
      public BeanTypeB a(){
        return new BeanTypeB();
      }
}
```

在上述例子中，类型不一致的 BeanTypeB 的实例，会被注入给变量 a，导致异常。

如果必须使用@Resource 注解，请指定 type 参数。如非必要，一般建议使用@Autowired 进行注入。

5 集合处理

5.1 【强制】关于 hashCode 和 equals 的处理，遵循如下规则：

- 只要重写 equals ，就必须重写 hashCode。
- 因为 Set 存储的是不重复的对象，依据 hashCode 和 equals 进行判断，所以 Set 存储的对象必须重写这两个方法。
- 如果自定义对象作为 Map 的键，那么必须重写 hashCode 和 equals 。

检查方式：插件扫描

说明：String 重写了 hashCode 和 equals 方法，所以我们可以非常愉快地使用 String 对象作为 key 来使用。

5.2 【强制】 ArrayList 的 subList 结果不可强转成 ArrayList ， 否则会抛出 ClassCastException 异常，即 java.util.RandomAccessSubList cannot be cast to java.util.ArrayList 。

检查方式：插件扫描

说明：subList 返回的是 ArrayList 的内部类 SubList ，并不是 ArrayList 而是 ArrayList 的一个视图，对于 SubList 子列表的所有操作最终会反映到原列表上。

5.3 【强制】在 subList 场景中，高度注意对原集合元素的增加或删除，均会导致子列表的遍历、增加、删除产生 ConcurrentModificationException 异常。

检查方式：插件扫描

5.4 【强制】使用集合转数组的方法，必须使用集合的 toArray(T[] array)。

检查方式：插件扫描

说明：使用 toArray 带参方法，入参分配的数组空间不够大时， toArray 方法内部将重新分配内存空间，并返回新数组地址 ； 如果数组元素个数大于实际所需，下标为 [list.size()]的数组元素将被置为 null ，其它数组元素保持原值，因此应将方法入参数组的大小定义成与集合元素的个数一致。

正例：

```
List<String> list = new ArrayList<String>(2);
list.add("guan");
list.add("bao");
```

```
String[] array = new String[list.size()];
array = list.toArray(array);
```

反例：直接使用 toArray 无参方法存在问题，此方法返回值只能是 Object[] 类，若强转其它类型数组将出现 ClassCastException 错误。

5.5【强制】使用工具类 Arrays.asList() 把数组转换成集合时，不能使用其修改集合相关的方法，它的 add / remove / clear 方法会抛出 UnsupportedOperationException 异常。

检查方式：插件扫描

说明：asList 的返回对象是一个 Arrays 内部类， 并没有实现集合的修改方法。 Arrays . asList 体现的是适配器模式，只是转换接口，后台的数据仍是数组。

```
String[] str = new String[] { "you", "wu" };
List list = Arrays.asList(str);
```

第一种情况： list.add("yangguanbao"); 运行时异常。

第二种情况： str[0] = "gujin"; 那么 list.get(0) 也会随之修改。

5.6【强制】泛型通配符<? extends T >来接收返回的数据，此写法的泛型集合不能使用 add 方法，而 <? super T> 不能使用 get 方法，作为接口调用赋值时易出错。

检查方式：插件扫描

说明：扩展说一下 PECS(Producer Extends Consumer Super) 原则：第一、频繁往外读取内容的，适合用<? extends T >。第二、经常往里插入的，适合用 <? super T> 。

5.7【强制】不要在 foreach 循环里进行元素的 remove / add 操作。remove 元素请使用 Iterator 方式，如果并发操作，需要对 Iterator 对象加锁。

检查方式：插件扫描

正例：

```
List<String> list = new ArrayList<>();
list.add("1");
list.add("2");
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    String item = iterator.next();
    if (删除元素的条件) {
```

```
            iterator.remove();
        }
    }
```

反例：

```
for (String item : list) {
    if ("1".equals(item)) {
        list.remove(item);
    }
}
```

5.8【强制】 在 JDK 7 版本及以上， Comparator 实现类要满足如下三个条件，不然 Arrays . sort ，Collections . sort 会报 IllegalArgumentException 异常。

检查方式：插件扫描

说明：三个条件如下

- x ， y 的比较结果和 y ， x 的比较结果相反。
- x > y ， y > z ，则 x > z 。
- x = y ，则 x ， z 比较结果和 y ， z 比较结果相同。

反例：下例中没有处理相等的情况，实际使用中可能会出现异常：

```
new Comparator<Student>() {
    @Override
    public int compare(Student o1, Student o2) {
        return o1.getId() > o2.getId() ? 1 : -1;
    }
};
```

5.9【推荐】集合泛型定义时，在 JDK7 及以上，使用 diamond 语法或全省略。

检查方式：人工检查

说明：菱形泛型，即 diamond ，直接使用<>来指代前边已经指定的类型。

正例：

```
// <> diamond 方式
HashMap<String, String> userCache = new HashMap<>(16);
// 全省略方式
ArrayList<User> users = new ArrayList(10);
```

5.10 【推荐】集合初始化时，指定集合初始值大小。

检查方式：插件扫描

说明：HashMap 使用 HashMap(int initialCapacity) 初始化。

正例：initialCapacity = (需要存储的元素个数 / 负载因子) + 1。 注意负载因子 （即 loaderfactor）默认为 0.75， 如果暂时无法确定初始值大小，请设置为 16（即默认值）。

反例：HashMap 需要放置 1024 个元素，由于没有设置容量初始大小，随着元素不断增加，容量 7 次被迫扩大， resize 需要重建 hash 表，严重影响性能。

5.11 【推荐】使用 entrySet 遍历 Map 类集合 KV ，而不是 keySet 方式进行遍历。

检查方式：插件扫描

说明：keySet 其实是遍历了 2 次，一次是转为 Iterator 对象，另一次是从 hashMap 中取出 key 所对应的 value 。而 entrySet 只是遍历了一次就把 key 和 value 都放到了 entry 中，效率更高。如果是 JDK 8，考虑到 Map . foreach 的遍历方式性能不佳，不推荐使用。

正例：values() 返回的是 V 值集合，是一个 list 集合对象 ；keySet() 返回的是 K 值集合，是一个 Set 集合对象 ；entrySet() 返回的是 K-V 值组合集合。

5.12 【推荐】高度注意 Map 类集合 K / V 能不能存储 null 值的情况，如下表格：

集合类	Key	Value	Super	说明
Hashtable	不允许为 null	不允许为 null	Dictionary	线程安全
ConcurrentHashMap	不允许为 null	不允许为 null	AbstractMap	锁分段技术（JDK8:CAS）
TreeMap	不允许为 null	允许为 null	AbstractMap	线程不安全
HashMap	允许为 null	允许为 null	AbstractMap	线程不安全

检查方式：无需检查

反例：由于 HashMap 的干扰，很多人认为 ConcurrentHashMap 是可以

置入 null 值，而事实上，存储 null 值时会抛出 NPE 异常。

5.13 【参考】合理利用好集合的有序性（sort）和稳定性（order），避免集合的无序性（unsort）和不稳定性（unorder）带来的负面影响。

检查方式：无需检查

说明：有序性是指遍历的结果是按某种比较规则依次排列的。稳定性指集合每次遍历的元素次序是一定的。如：ArrayList 是 order / unsort；HashMap 是 unorder / unsort；TreeSet 是 order / sort 。

5.14 【参考】利用 Set 元素唯一的特性，可以快速对一个集合进行去重操作，避免使用 List 的 contains 方法进行遍历、对比、去重操作。

检查方式：人工检查

6 并发处理

6.1 【强制】获取单例对象需要保证线程安全，其中的方法也要保证线程安全。

检查方式：人工检查

说明：资源驱动类、工具类、单例工厂类都需要注意。

6.2 【强制】创建线程或线程池时请指定有意义的线程名称，方便出错时回溯。

检查方式：插件扫描

正例：

```
public class TimerTaskThread extends Thread {
    public TimerTaskThread() {
        super.setName("TimerTaskThread");
        ...
    }
}
```

6.3 【强制】线程资源必须通过线程池提供，不允许在应用中自行显式创建线程。

检查方式：插件扫描

说明：使用线程池的好处是减少在创建和销毁线程上所消耗的时间以及系统资源的开销，解决资源不足的问题。如果不使用线程池，有可能造成系统创建大量同类线程而导致消耗完内存或者“过度切换”的问题。

6.4 【强制】线程池不允许使用 Executors 去创建，而是通过 ThreadPoolExecutor 的方式，这样的处理方式让写的同学更加明确线程池的运行规则，规避资源耗尽的风险。

检查方式：插件扫描

说明：Executors 返回的线程池对象的弊端如下：

FixedThreadPool 和 SingleThreadPool ：

允许的请求队列长度为 Integer.MAX_VALUE ，可能会堆积大量的请求，从而导致 OOM 。

CachedThreadPool 和 ScheduledThreadPool ：

允许的创建线程数量为 Integer.MAX_VALUE ，可能会创建大量的线程，从而导致 OOM 。

6.5【强制】 SimpleDateFormat 是线程不安全的类，一般不要定义为 static 变量，如果定义为 static ，必须加锁，或者使用 DateUtils 工具类。

检查方式：插件扫描

正例：注意线程安全，使用 DateUtils 。亦推荐如下处理：

```
    private static final ThreadLocal<DateFormat> df = new
ThreadLocal<DateFormat>() {
        @ Override
        protected DateFormat initialValue() {
            return new SimpleDateFormat("yyyy-MM-dd");
        }
    };
```

说明：如果是 JDK 8 的应用，可以使用 Instant 代替 Date ，LocalDateTime 代替 Calendar ，DateTimeFormatter 代替 SimpleDateFormat ，官方给出的解释： simple beautiful strongimmutable thread - safe 。

6.6【强制】高并发时，同步调用应该去考量锁的性能损耗。能用无锁数据结构，就不要用锁 ； 能锁区块，就不要锁整个方法体 ； 能用对象锁，就不要用类锁。

检查方式：人工检查

说明：尽可能使加锁的代码块工作量尽可能的小，避免在锁代码块中调用 RPC 方法。

6.7【强制】对多个资源、数据库表、对象同时加锁时，需要保持一致的加锁顺序，否则可能会造成死锁。

检查方式：人工检查

说明：线程一需要对表 A 、 B 、 C 依次全部加锁后才可以进行更新操

作，那么线程二的加锁顺序也必须是 A 、 B 、 C ，否则可能出现死锁。

6.8 【强制】并发修改同一记录时，避免更新丢失，需要加锁。要么在应用层加锁，要么在缓存加锁，要么在数据库层使用乐观锁，使用 version 作为更新依据。

检查方式：人工检查

说明：如果每次访问冲突概率小于 20%，推荐使用乐观锁，否则使用悲观锁。乐观锁的重试次数不得小于 3 次。

6.9 【强制】多线程并行处理定时任务时， Timer 运行多个 TimeTask 时，只要其中之一没有捕获抛出的异常， 其它任务便会自动终止运行， 使用 ScheduledExecutorService 则没有这个问题。

检查方式：人工检查

6.10 【推荐】使用 CountDownLatch 进行异步转同步操作，每个线程退出前必须调用 countDown 方法，线程执行代码注意 catch 异常，确保 countDown 方法被执行到，避免主线程无法执行至 await 方法，直到超时才返回结果。

检查方式：人工检查

说明：注意，子线程抛出异常堆栈，不能在主线程 try - catch 到。

6.11 【推荐】避免 Random 实例被多线程使用，虽然共享该实例是线程安全的，但会因竞争同一 seed 导致的性能下降。

检查方式：人工检查

说明： Random 实例包括 java . util . Random 的实例或者 Math . random() 的方式。

正例：在 JDK 7 之后，可以直接使用 API ThreadLocalRandom ，而在 JDK 7 之前，需要编码保证每个线程持有一个实例。

6.12 【推荐】在并发场景下，通过双重检查锁（double-checked locking）实现延迟初始化的优化问题隐患（ 可参考 The " Double - Checked Locking is Broken " Declaration) ，推荐解决方案中较为简单一种 （ 适用于 JDK 5 及以上版本 ） ，将目标属性声明为 volatile 型 。

检查方式：人工检查

反例：

```
class LazyInitDemo {
    private Helper helper = null;
    public Helper getHelper() {
```

```
            if (helper == null) synchronized(this) {
                if (helper == null)
                    helper = new Helper();
            }
            return helper;
        }
        // other methods and fields...
    }
```

6.13 【参考】 volatile 解决多线程内存不可见问题。对于一写多读，是可以解决变量同步问题，但是如果多写，同样无法解决线程安全问题。如果是 count ++操作，使用如下类实现：AtomicInteger count = new AtomicInteger(); count . addAndGet(1); 如果是 JDK 8，推荐使用 LongAdder 对象，比 AtomicLong 性能更好 （ 减少乐观锁的重试次数 ）。

检查方式：人工检查

6.14 【参考】 HashMap 在容量不够进行 resize 时由于高并发可能出现死链，导致 CPU 飙升，在开发过程中可以使用其它数据结构或加锁来规避此风险。

检查方式：人工检查

6.15 【推荐】 ThreadLocal 无法解决共享对象的更新问题，ThreadLocal 对象建议使用 static 修饰。这个变量是针对一个线程内所有操作共享的，所以设置为静态变量，所有此类实例共享此静态变量 ，也就是说在类第一次被使用时装载，只分配一块存储空间，所有此类的对象（ 只要是这个线程内定义的 ）都可以操控这个变量。

检查方式： 插件扫描

7 控制语句

7.1 【强制】在一个 switch 块内，每个 case 要么通过 break / return 等来终止，要么注释说明程序将继续执行到哪一个 case 为止 ； 在一个 switch 块内，都必须包含一个 default 语句并且放在最后，即使空代码。

检查方式：插件扫描

7.2 【强制】在 if / else / for / while / do 语句中必须使用大括号。即使只有一行代码，避免采用单行的编码方式： if (condition) statements;

检查方式：插件扫描

7.3 【强制】在高并发场景中，避免使用”等于”判断作为中断或退出的条

件。

检查方式：人工检查

说明：如果并发控制没有处理好，容易产生等值判断被“击穿”的情况，使用大于或小于的区间判断条件来代替。

反例：判断剩余奖品数量等于 0 时，终止发放奖品，但因为并发处理错误导致奖品数量瞬间变成了负数，这样的话，活动无法终止。

7.4【推荐】表达异常的分支时，少用 if-else 方式，这种方式可以改写成：

```
if (condition) {
    ...
    return obj;
}
// 接着写 else 的业务逻辑代码;
```

检查方式：人工检查

说明：如果非得使用 if()...else if()...else... 方式表达逻辑，【强制】避免后续代码维护困难，请勿超过 3 层。

正例： 超过 3 层的 if-else 的逻辑判断代码可以使用卫语句、 策略模式、 状态模式等来实现，其中卫语句示例如下：

```
public void today() {
    if (isBusy()) {
        System.out.println(“change time.”);
        return;
    }
    if (isFree()) {
        System.out.println(“go to travel.”);
        return;
    }
    System.out.println(“stay at home to learn hundsun Java
Coding Guidelines.”);
    return;
}
```

7.5【推荐】除常用方法（如 getXxx/isXxx ）等外，不要在条件判断中执行其它复杂的语句，将复杂逻辑判断的结果赋值给一个有意义的布尔变量名，以提高可读性。

检查方式：插件扫描

正例：

```
// 伪代码如下
final boolean existed = (file.open(fileName, "w") != null) &&
(...) || (...);
if (existed) {
    ...
}
```

反例：

```
if ((file.open(fileName, "w") != null) && (...) || (...)) {
    ...
}
```

7.6【推荐】在循环体内，避免将 Try-Catch 的异常分支，当作正常业务逻辑使用，Catch 分支只用于处理非期望的异常场景。而在正常业务处理中，无论业务参数为何，都不应主动进入 Catch 分支。

检查方式：人工检查

反例：

```
    for (int i=0;i<fileLines.size();i++){
            try{
                // ---一般业务逻辑处理--
                if ("ABC".equals(fileLines.get(i).getBank())) {
                    throw new ABCLogicException();
                }else if
("ICBC".equals(fileLines.get(i).getBank())) {
                    throw new ICBCLogicException();
                }
        // ---一般业务逻辑处理--
            }catch (ABCLogicException e){
                // --农行业务逻辑处理--
            }catch (ICBCLogicException e){
                // --工行业务逻辑处理--
            }
        }
```

解释：上述例子中，无论农行业务逻辑还是工行业务逻辑，都属于正常

的业务逻辑，不应用抛异常捕获的方式去处理。抛异常容易让阅读代码的人误解这些代码的含义。另外，如果是循环内频繁进入异常分支，一定程度上也会影响性能。上述场景，推荐通过方法调用来处理。

正例：

```
    for (int i=0;i<fileLines.size();i++){
            try{
                // ---一般业务逻辑处理--
                if ("ABC".equals(fileLines.get(i).getBank())) {
          // 调用农行业务逻辑处理方法
                    doABCLogic();
                }else if
("ICBC".equals(fileLines.get(i).getBank())) {
          // 调用工行业务逻辑处理方法
                    doICBCLogic();
                }
        // ---一般业务逻辑处理--
            }catch (BusinessException e){
                // --业务异常处理--
            }
        }
```

7.7 【参考】避免采用取反逻辑运算符。

检查方式：插件扫描

说明：取反逻辑不利于快速理解，并且取反逻辑写法必然存在对应的正向逻辑写法。

正例：使用 if (x < 628) 来表达 x 小于 628。

反例：使用 if (!(x >= 628)) 来表达 x 小于 628。

7.8 【推荐】接口入参保护，这种场景常见的是用作批量操作的接口。

检查方式：人工检查

7.9 【参考】下列情形，需要进行参数校验：

- 调用频次低的方法。
- 执行时间开销很大的方法。此情形中，参数校验时间几乎可以忽略不计，但如果因为参数错误导致中间执行回退，或者错误，那得不偿失。

- 需要极高稳定性和可用性的方法。
- 对外提供的开放接口，不管是 RPC / API / HTTP 接口。
- 敏感权限入口。

检查方式：人工检查

7.10 【参考】下列情形，不需要进行参数校验：

- 极有可能被循环调用的方法。但在方法说明里必须注明外部参数检查要求。
- 底层调用频度比较高的方法。毕竟是像纯净水过滤的最后一道，参数错误不太可能到底层才会暴露问题。一般 DAO 层与 Service 层都在同一个应用中，部署在同一台服务器中，所以 DAO 的参数校验，可以省略。
- 被声明成 private 只会被自己代码所调用的方法，如果能够确定调用方法的代码传入参数已经做过检查或者肯定不会有问题，此时可以不校验参数。

检查方式：人工检查

7.11 【强制】表达异常的分支时，少用 if-else 方式。超过 3 层的 if-else 的逻辑判断代码可以使用卫语句、策略模式、状态模式等来实现。

检查方式：插件扫描

7.12 【强制】禁止超过 3 层以上的 try 嵌套

检查方式：插件扫描

反例：

```
try {
    try {
        try {
            try {
                try { // violation, current depth is 4, max allowed depth is 3
                } catch (Exception e) {
                }
            } catch (Exception e) {
            }
        } catch (Exception e) {
    }
```

```
                } catch (Exception e) {
            }
        } catch (Exception e) {
    }
```

7.13 【强制】禁止超过 3 层以上的 for 嵌套

检查方式：插件扫描

正例：

```
for(int i=0; i<10; i++) {
    for(int j=0; j<i; j++) {
        for(int k=0; k<j; k++) { // ok
        }
    }
}
```

反例：

```
for(int i=0; i<10; i++) {
    for(int j=0; j<i; j++) {
        for(int k=0; k<j; k++) {
            for(int l=0; l<k; l++)
{ // violation, max allowed nested loop number is 2
            }
        }
    }
}
```

8 注释规约

8.1 【强制】类、类属性、类方法的注释必须使用 Javadoc 规范，使用/**内容*/格式，不得使用// xxx 方式。

检查方式：插件扫描

说明：在 IDE 编辑窗口中， Javadoc 方式会提示相关注释，生成 Javadoc 可以正确输出相应注释 ； 在 IDE 中，工程调用方法时，不进入方法即可悬浮提示方法、参数、返回值的意义，提高阅读效率。

8.2 【强制】所有的抽象方法（ 包括接口中的方法 ）必须要用 Javadoc 注释、除了返回值、参数、异常说明外，还必须指出该方法做什么事情，实现什么功能。

检查方式：插件扫描

说明：对子类的实现要求，或者调用注意事项，请一并说明。

8.3【强制】所有的类都必须添加创建者和创建日期。

检查方式：插件扫描

8.4【推荐】方法内部单行注释，在被注释语句上方另起一行，使用//注释。方法内部多行注释使用/* */注释，注意与代码对齐。

检查方式：插件扫描

8.5【强制】所有的枚举类型字段必须要有注释，说明每个数据项的用途。

检查方式：插件扫描

8.6【推荐】与其“半吊子”英文来注释，不如用中文注释把问题说清楚。专有名词与关键字保持英文原文即可。

检查方式：人工检查

反例：“ TCP 连接超时”解释成“传输控制协议连接超时”，理解反而费脑筋。

8.7【推荐】代码修改的同时，注释也要进行相应的修改，尤其是参数、返回值、异常、核心逻辑等的修改。

检查方式：人工检查

说明：代码与注释更新不同步， 就像路网与导航软件更新不同步一样，如果导航软件严重滞后，就失去了导航的意义。

8.8【参考】谨慎注释掉代码。在上方详细说明，而不是简单地注释掉。如果无用，则删除。

检查方式：人工检查

说明：代码被注释掉有两种可能性：1 ） 后续会恢复此段代码逻辑。2 ）永久不用。前者如果没有备注信息，难以知晓注释动机。后者建议直接删掉 （ 代码仓库保存了历史代码 ） 。

8.9【参考】对于注释的要求：第一、能够准确反应设计思想和代码逻辑 ；第二、能够描述业务含义，使别的程序员能够迅速了解到代码背后的信息。完全没有注释的大段代码对于阅读者形同天书，注释是给自己看的，即使隔很长时间，也能清晰理解当时的思路 ； 注释也是给继任者看的，使其能够快速接替自己的工作。

检查方式：无需检查

8.10【参考】好的命名、代码结构是自解释的，注释力求精简准确、表达到位。避免出现注释的一个极端：过多过滥的注释，代码的逻辑一旦修改，修改注释是相当大的负担。

检查方式：人工检查

反例：

```
// put elephant into fridge
put(elephant, fridge);
```

方法名 put ，加上两个有意义的变量名 elephant 和 fridge ，已经说明了这是在干什么，语义清晰的代码不需要额外的注释。

8.11 【参考】 特殊注释标记， 请注明标记人与标记时间。 注意及时处理这些标记， 通过标记扫描，经常清理此类标记。线上故障有时候就是来源于这些标记处的代码。

检查方式：无需检查

- 待办事宜 （TODO） ：（ 标记人，标记时间， [预计处理时间]）

  表示需要实现， 但目前还未实现的功能。 这实际上是一个 Javadoc 的标签， 目前的 Javadoc 还没有实现， 但已经被广泛使用。 只能应用于类， 接口和方法 （ 因为它是一个 Javadoc 标签 ）。

- 错误，不能工作 （FIXME） ：（ 标记人，标记时间， [预计处理时间]）

  在注释中用 FIXME 标记某代码是错误的，而且不能工作，需要及时纠正的情况。

9 异常处理

9.1 【强制】Java 类库中定义的可以通过预检查方式规避的 RuntimeException 异常不应该通过 catch 的方式来处理，比如：NullPointerException ， IndexOutOfBoundsException 等 Java 原生运行时异常。

检查方式：插件扫描

说明：无法通过预检查的异常除外，比如，在解析字符串形式的数字时，不得不通过 catchNumberFormatException 来实现。

正例：if (obj != null) {...}

反例：try { obj.method(); } catch (NullPointerException e) {…}

9.2 【强制】异常不要用来做流程控制，条件控制。

检查方式：人工检查

说明：异常设计的初衷是解决程序运行中的各种意外情况，且异常的处理效率比条件判断方式要低很多。

9.3 【强制】 catch 时请分清稳定代码和非稳定代码，稳定代码指的是无论如何不会出错的代码。对于非稳定代码的 catch 尽可能进行区分异常类型，再做对应的异常处理。

检查方式：人工检查

说明：对大段代码进行 try - catch ，使程序无法根据不同的异常做出正确的应激反应，也不利于定位问题，这是一种不负责任的表现。

正例：用户注册的场景中，如果用户输入非法字符，或用户名称已存在，或用户输入密码过于简单，在程序上作出分门别类的判断，并提示给用户。

9.4【强制】捕获异常是为了处理它，不要捕获了却什么都不处理而抛弃之，如果不想处理它，请将该异常抛给它的调用者。最外层的业务使用者，必须处理异常，将其转化为用户可以理解的内容。

检查方式：插件扫描

9.5【强制】有 try 块放到了事务代码中， catch 异常后，如果需要回滚事务，一定要注意手动回滚事务。

检查方式：人工检查

9.6【强制】 finally 块必须对资源对象、流对象进行关闭，有异常也要做 try - catch 。

检查方式：人工检查

说明：如果 JDK 7 及以上，可以使用 try - with - resources 方式。

9.7【强制】不要在 finally 块中使用 return 。

检查方式：插件扫描

说明： finally 块中的 return 返回后方法结束执行，不会再执行 try 块中的 return 语句。

9.8【强制】捕获异常与抛异常，必须是完全匹配，或者捕获异常是抛异常的父类。

检查方式：人工检查

说明：如果预期对方抛的是绣球，实际接到的是铅球，就会产生意外情况。

9.9【推荐】方法的返回值可以为 null ，不强制返回空集合，或者空对象等，必须添加注释充分说明什么情况下会返回 null 值。

检查方式：人工检查

说明：本文档明确防止 NPE 是调用者的责任。即使被调用方法返回空集合或者空对象，对调用者来说，也并非高枕无忧，必须考虑到远程调用失败、序列化失败、运行时异常等场景返回 null 的情况。

9.10【推荐】防止 NPE ，是程序员的基本修养，注意 NPE 产生的场景：

- 返回类型为基本数据类型， return 包装数据类型的对象时，自动拆箱有可能产生 NPE 。

反例：public int f() { return Integer 对象}， 如果为 null ，自动解箱抛 NPE 。

- 数据库的查询结果可能为 null 。
- 集合里的元素即使 isNotEmpty ，取出的数据元素也可能为 null 。
- 远程调用返回对象时，一律要求进行空指针判断，防止 NPE 。
- 对于 Session 中获取的数据，建议 NPE 检查，避免空指针。
- 级联调用 obj . getA() . getB() . getC()； 一连串调用，易产生 NPE 。

检查方式：无需检查

正例：使用 JDK8 的 Optional 类来防止 NPE 问题。

9.11 【推荐】 定义时区分 unchecked / checked 异常， 避免直接抛出 new RuntimeException() ，更不允许抛出 Exception 或者 Throwable ， 应使用有业务含义的自定义异常。 推荐业界已定义过的自定义异常，如： DAOException / ServiceException 等。

检查方式：无需检查

9.12 【参考】对于公司外的 http / api 开放接口必须使用“错误码” ；而应用内部推荐异常抛出 ；跨应用间 RPC 调用优先考虑使用 Result 方式，封装 isSuccess()方法 、“错误码”、“错误简短信息”。

检查方式：人工检查

说明：关于 RPC 方法返回方式使用 Result 方式的理由：

- 使用抛异常返回方式，调用方如果没有捕获到就会产生运行时错误。
- 如果不加栈信息，只是 new 自定义异常，加入自己的理解的 error message ，对于调用端解决问题的帮助不会太多。如果加了栈信息，在频繁调用出错的情况下，数据序列化和传输的性能损耗也是问题。

9.13 【参考】避免出现重复的代码 （Don ’ t Repeat Yourself） ，即 DRY 原则。

检查方式：人工检查

说明：随意复制和粘贴代码，必然会导致代码的重复，在以后需要修改时，需要修改所有的副本，容易遗漏。必要时抽取共性方法，或者抽象公共类，甚至是组件化。

正例：一个类中有多个 public 方法， 都需要进行数行相同的参数校验操作， 这个时候请抽取：

```
private boolean checkParam(DTO dto) {...}
```

10 日志规约

10.1 【推荐】应用中不建议直接使用日志系统 （Log4j、 Logback） 中的 API ，建议使用门面模式的日志框架（如 SLF4J ）有利于维护和各个类的日志处理方式统一。

检查方式：人工检查

正例：
```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
private static final Logger logger =
LoggerFactory.getLogger(Abc.class);
```

10.2 【强制】日志文件至少保存 15 天，因为有些异常具备以“周”为频次发生的特点。

检查方式：人工检查

10.3 【强制】应用中的扩展日志 （ 如打点、临时监控、访问日志等 ） 命名方式：

appName_logType_logName.log 。

检查方式：人工检查

logType :日志类型，如 stats / monitor / access 等 ；logName :日志描述。这种命名的好处：通过文件名就可知道日志文件属于什么应用，什么类型，什么目的，也有利于归类查找。

正例：mppserver 应用中单独监控时区转换异常，如：

mppserver _ monitor _ timeZoneConvert . log

说明：推荐对日志进行分类，如将错误日志和业务日志分开存放，便于开发人员查看，也便于通过日志对系统进行及时监控。

10.4 【强制】对 trace / debug / info 级别的日志输出，必须使用条件输出形式或者使用占位符的方式。

检查方式：插件扫描

说明：logger . debug(″ Processing trade with id : ″ + id + ″ and symbol : ″ + symbol);

如果日志级别是 warn ， 上述日志不会打印， 但是会执行字符串拼接操作， 如果 symbol 是对象，会执行 toString() 方法，浪费了系统资源，执行了上述操作，最终日志却没有打印。

正例：（条件）建设采用如下方式

```
if (logger.isDebugEnabled()) {
    logger.debug("Processing trade with id: " + id + " and symbol: " + symbol);
}
```

正例：（占位符）

```
logger.debug("Processing trade with id: {} and symbol : {} ", id, symbol);
```

10.5 【强制】避免重复打印日志，浪费磁盘空间，务必在 log4j.xml 中设置 additivity = false 。

检查方式：人工检查

正例：<logger name="com.taobao.dubbo.config" additivity="false">

10.6 【强制】异常信息应该包括两类信息：案发现场信息和异常堆栈信息。如果不处理，那么通过关键字 throws 往上抛出。

检查方式：人工检查

正例： logger.error("修改证券信息失败，证券信息={}", stockInfo, e); //需要确保 stockInfo.toString()方法已经实现，不建议显式调用 toString 方法，避免日志不输出时（一般 Info 级别以上日志可能会不输出），还会调用 toString 方法，占用业务处理时间。

10.7 【推荐】谨慎地记录日志。生产环境禁止输出 debug 日志 ； 有选择地输出 info 日志 ； 如果使用 warn 来记录刚上线时的业务行为信息，一定要注意日志输出量的问题，避免把服务器磁盘撑爆，并记得及时删除这些观察日志。

检查方式：人工检查

说明：大量地输出无效日志，不利于系统性能提升，也不利于快速定位错误点。记录日志时需关注采集的必要性和实用性。

10.8 【推荐】可以使用 warn 日志级别来记录用户输入参数错误的情况，如非必要，请不要在此场景打出 error 级别，避免频繁报警。

检查方式：人工检查

说明：注意日志输出的级别，error 级别只记录系统逻辑出错、异常或者重要的错误信息。

10.9 【推荐】尽量用英文来描述日志错误信息，如果日志中的错误信息用英文描述不清楚的话使用中文描述即可， 否则容易产生歧义。 国际

化团队或海外部署的服务器由于字符集问题。

【强制】使用全英文来注释和描述日志错误信息。

检查方式：人工检查

11 二方库依赖

11.1 【强制】定义 GAV 遵从以下规则：

- G GroupID 格式： com.{公司/BU}.业务线[.子业务线]，最多 4 级。

说明：{公司/BU} 例如： hundsun / hscloud / hs 等 BU 一级； 子业务线可选。

正例：com.hundsun.platform 或 com.hundsun.jres.workflow

- A ArtifactID 格式：产品线名-模块名。语义不重复不遗漏，先到中央仓库去查证一下。

正例：jresplus-remoting / jresplus-ui-runtime / jresplus-cep-core

- V Version ：详细规定参考下方。

检查方式：插件扫描

11.2 【强制】二方库版本号命名方式：主版本号.次版本号.修订号

- 主版本号：产品方向改变，或者大规模 API 不兼容，或者架构不兼容升级。
- 次版本号：保持相对兼容性，增加主要功能特性，影响范围极小的 API 不兼容修改。
- 修订号：保持完全兼容性，修复 BUG 、新增次要功能特性等。

检查方式：人工检查

说明：注意起始版本号必须为： 1.0.0 ，而不是 0.0.1 正式发布的类库必须先去中央仓库进行查证，使版本号有延续性，正式版本号不允许覆盖升级。如当前版本：1.3.3，那么下一个合理的版本号：1.3.4 或 1.4.0 或 2.0.0

11.3 【强制】线上应用不要依赖 SNAPSHOT 版本 （ 安全包除外 ）。

检查方式：人工检查

说明：不依赖 SNAPSHOT 版本是保证应用发布的幂等性。 另外， 也可以加快编译时的打包构建。

11.4 【强制】二方库的新增或升级，保持除功能点之外的其它 jar 包仲裁结果不变。如果有改变，必须明确评估和验证，建议进行 dependency：resolve 前后信息比对，如果仲裁结果完全不一致，那么通过 dependency：tree 命令，找出差异点，进行<excludes>排除 jar 包。

检查方式：人工检查

11.5 【强制】二方库里可以定义枚举类型，参数可以使用枚举类型，但是接口返回值不允许使用枚举类型或者包含枚举类型的 POJO 对象。

检查方式：人工检查

11.6 【强制】依赖于一个二方库群时，必须定义一个统一的版本变量，避免版本号不一致。

检查方式：人工检查

说明：依赖 springframework - core ,- context ,- beans ，它们都是同一个版本，可以定义一个变量来保存版本：${spring.version}，定义依赖的时候，引用该版本。

11.7 【强制】禁止在子项目的 pom 依赖中出现相同的 GroupId ，相同的 ArtifactId ，但是不同的 Version 。

检查方式：

说明：在本地调试时会使用各子项目指定的版本号，但是合并成一个 war ，只能有一个版本号出现在最后的 lib 目录中。可能出现线下调试是正确的，发布到线上却出故障的问题。

11.8 【推荐】所有 pom 文件中的依赖声明放在< dependencies >语句块中，所有版本仲裁放在< dependencyManagement >语句块中。

检查方式：人工检查

说明：< dependencyManagement >里只是声明版本，并不实现引入，因此子项目需要显式的声明依赖， version 和 scope 都读取自父 pom 。而< dependencies >所有声明在主 pom 的< dependencies >里的依赖都会自动引入，并默认被所有的子项目继承。

11.9 【推荐】二方库不要有配置项，最低限度不要再增加配置项。

检查方式：人工检查

11.10 【参考】为避免应用二方库的依赖冲突问题，二方库发布者应当遵循以下原则：

- 精简可控原则。移除一切不必要的 API 和依赖，只包含 Service API 、必要的领域模型对象、 Utils 类、常量、枚举等。如果依赖其它二方库，尽量是 provided 引入，让二方库使用者去依赖具体版本号 ； 无 log 具体实现，只依赖日志框架。
- 稳定可追溯原则。每个版本的变化应该被记录，二方库由谁维护，源码在哪里，都需要能方便查到。除非用户主动升级

版本，否则公共二方库的行为不应该发生变化。

检查方式：人工检查

12 其它

12.1 【强制】在使用正则表达式时，利用好其预编译功能，可以有效加快正则匹配速度。

检查方式：人工检查

说明：不要在方法体内定义： Pattern pattern = Pattern . compile(“ 规则 ”);

12.2 【强制】 velocity 调用 POJO 类的属性时，建议直接使用属性名取值即可，模板引擎会自动按规范调用 POJO 的 getXxx() ，如果是 boolean 基本数据类型变量 （boolean 命名不需要加 is 前缀 ），会自动调用 isXxx() 方法。

检查方式：插件扫描

说明：注意如果是 Boolean 包装类对象，优先调用 getXxx() 的方法。

12.3 【强制】后台输送给页面的变量必须加 $!{var} ——中间的感叹号。

检查方式：插件扫描

说明：如果 var 等于 null 或者不存在，那么 ${var} 会直接显示在页面上。

12.4 【强制】注意 Math . random() 这个方法返回是 double 类型，注意取值的范围 0≤ x <1 （ 能够取到零值，注意除零异常 ），如果想获取整数类型的随机数，不要将 x 放大 10 的若干倍然后取整，直接使用 Random 对象的 nextInt 或者 nextLong 方法。

检查方式：插件扫描

12.5 【强制】获取当前毫秒数 System . currentTimeMillis(); 而不是 new Date() . getTime();

检查方式：插件扫描

说明：如果想获取更加精确的纳秒级时间值，使用 System. nanoTime() 的方式 。在 JDK 8 中，针对统计时间等场景，推荐使用 Instant 类。

12.6 【推荐】任何数据结构的构造或初始化，都应指定大小，避免数据结构无限增长吃光内存。

检查方式：插件扫描

12.7 【推荐】及时清理不再使用的代码段或配置信息。

检查方式：插件扫描

说明：对于垃圾代码或过时配置，坚决清理干净，避免程序过度臃肿，

代码冗余。

正例：对于暂时被注释掉，后续可能恢复使用的代码片断，在注释代码上方，统一规定使用三个斜杠(///)来说明注释掉代码的理由。

12.8 【参考】分层领域模型规约：

- DO（Data Object）：此对象与数据库表结构一一对应，通过 DAO 层向上传输数据源对象。
- DTO（Data Transfer Object）：数据传输对象， Service 或 Manager 向外传输的对象。
- BO（Business Object）：业务对象，由 Service 层输出的封装业务逻辑的对象。
- AO （Application Object）： 应用对象， 在 Web 层与 Service 层之间抽象的复用对象模型，极为贴近展示层，复用度不高。
- VO（View Object）：显示层对象，通常是 Web 向模板渲染引擎层传输的对象。
- Query：数据查询对象，各层接收上层的查询请求。注意超过 2 个参数的查询封装，禁止使用 Map 类来传输。

12.9 【强制】服务使用 JRES 服务支持的参数。

检查方式：插件扫描

说明：service 接口方法必须使用包装类型

12.10 【强制】服务参数（JresCloudService）应尽量使用单个自定义的 JavaBean，将实际传参封装在 JavaBean 内（在必须使用多个的情况下，应加上注解@CloudFunctionParam）

检查方式：插件扫描

说明：在 JRES 框架下，服务参数使用多个参数，可以被正常识别。但如果切换成 JRES for Springcloud 框架，则无法识别。因为 Springcloud 在不加注解的情况下，只能识别单个入参。为了确保框架切换的兼容性，建议服务的入参尽量为单个自定义的 JavaBean。

正例：

```
GetWorkDateResult getWorkDate(GetWorkDateParam param)
```

反例：

```
GetWorkDateResult getWorkDate(String contry, String tenantId, Integer
```

```
dealDate, String offSetFlag)
```

12.11 【强制】日期格式转换禁止使用大写 YYYY（即 new SimpleDateFormat(pattern)中 pattern 不能用 YYYY）

检查方式：插件扫描

说明：在日期格式里，yyyy 是我们通常所用的时间，而 YYYY 则是 week year（当前周所包含的年份的较大值），正常情况下二者是一样的，但是在 12 月最后一周时，可能出现跨年导致得到的年+1

```
SimpleDateFormat format = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
SimpleDateFormat format1 = new SimpleDateFormat("YYYY-MM-dd HH:mm:ss");
try {
    System.out.println(format.format(format.parse("2021-12-31 12:00:00")));
    System.out.println(format1.format(format.parse("2021-12-31
12:00:00")));
} catch (ParseException e) {
    e.printStackTrace();
}
```

如上示例输出分别是 2021-12-31 12:00:00 和 2022-12-31 12:00:00

12.12 【推荐】基于 JresCloud 框架开发的产品，业务代码禁用 System.setProperty 方法使用，而应采用 ConfigUtils 提供的对应方法替代使用。

检查方式：插件扫描

说明：优化 configutils 读取系统变量时的效率（注：因系统变量读取存在锁的问题，启动时拷贝系统变量为 Map 放入环境变量中，因此运行中使用 System.setProperty()修改或添加系统变量属性需要使用 System.getProperty()获取，无法再使用 ConfigUtils.get()获取到修改）

五、 IDE 配置项设置

1 文件注释

新建文件时需要补充标签 Description 的内容，为当前文件的主要作用；修

改文件时，需要修改 UpdateUser、UpdateDate、UpdateRemark 三个标签内容。

文件头注释设置：

Eclipse 中首选项->java->Code Style->Code Templates 中”Comments->Files”添加以下内容：

```
/**
 * 文件描述
 * @ProductName:    Hundsun HEP
 * @ProjectName:    ${PROJECT_NAME}
 * @Package:        ${PACKAGE_NAME}
 * @Description:    note
 * @Author:         ${USER}
 * @CreateDate:     ${DATE} ${TIME}
 * @UpdateUser:     ${USER}
 * @UpdateDate:     ${DATE} ${TIME}
 * @UpdateRemark:   The modified content
 * @Version:        1.0
 *
 * Copyright © ${YEAR} Hundsun Technologies Inc. All Rights Reserved
**/
```

说明：eclipse 中也可以导入文件注释模板 codetemplates.xml

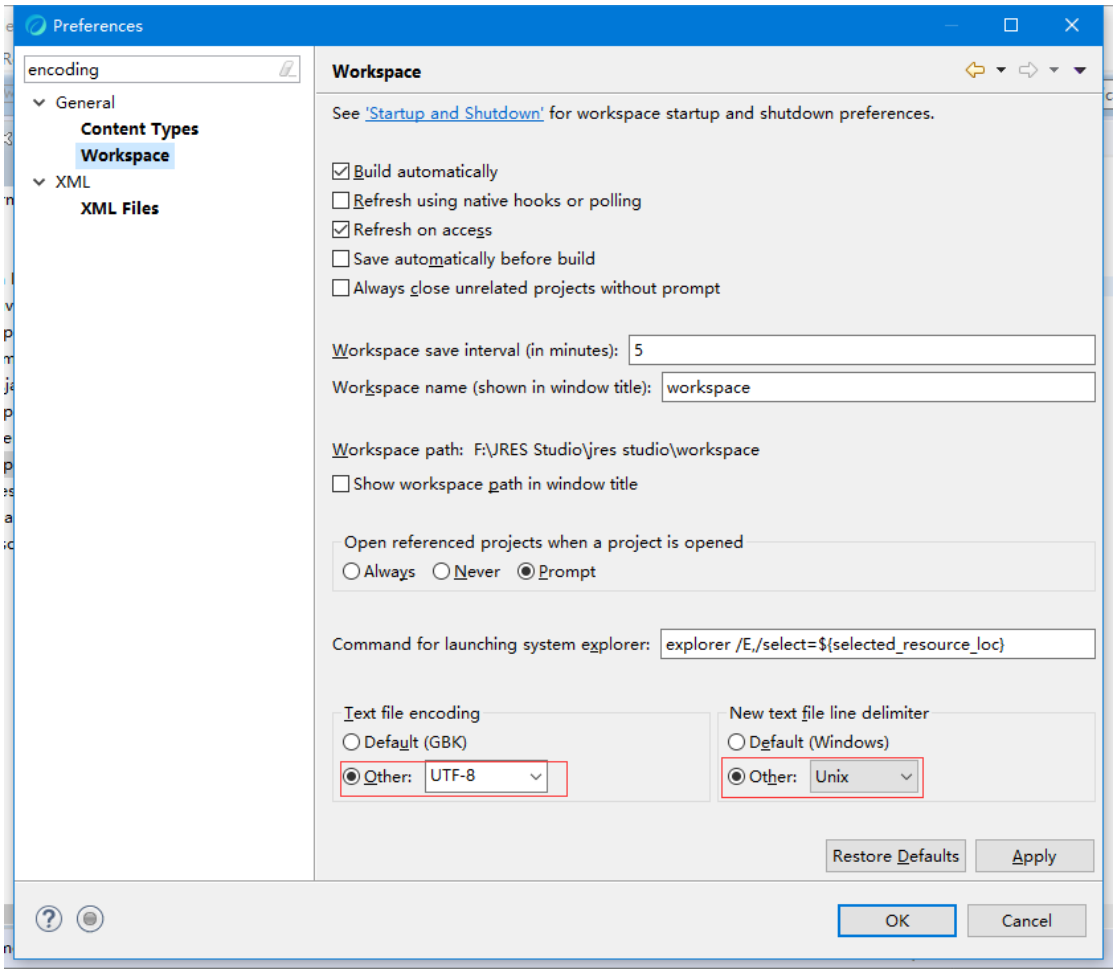
IntelljIDEA 在 File > Settings > Editor > Copyright > Copyright Profiles 中添加以下内容：

```
/**
 * 文件描述
 * @ProductName:    Hundsun HEP
 * @ProjectName:    ${PROJECT_NAME}
 * @Package:        ${PACKAGE_NAME}
 * @Description:    note
 * @Author:         ${USER}
 * @CreateDate:     ${DATE} ${TIME}
 * @UpdateUser:     ${USER}
 * @UpdateDate:     ${DATE} ${TIME}
```

```
 * @UpdateRemark:   The modified content
 * @Version:        1.0
 *
 * Copyright © ${YEAR} Hundsun Technologies Inc. All Rights Reserved
**/
```

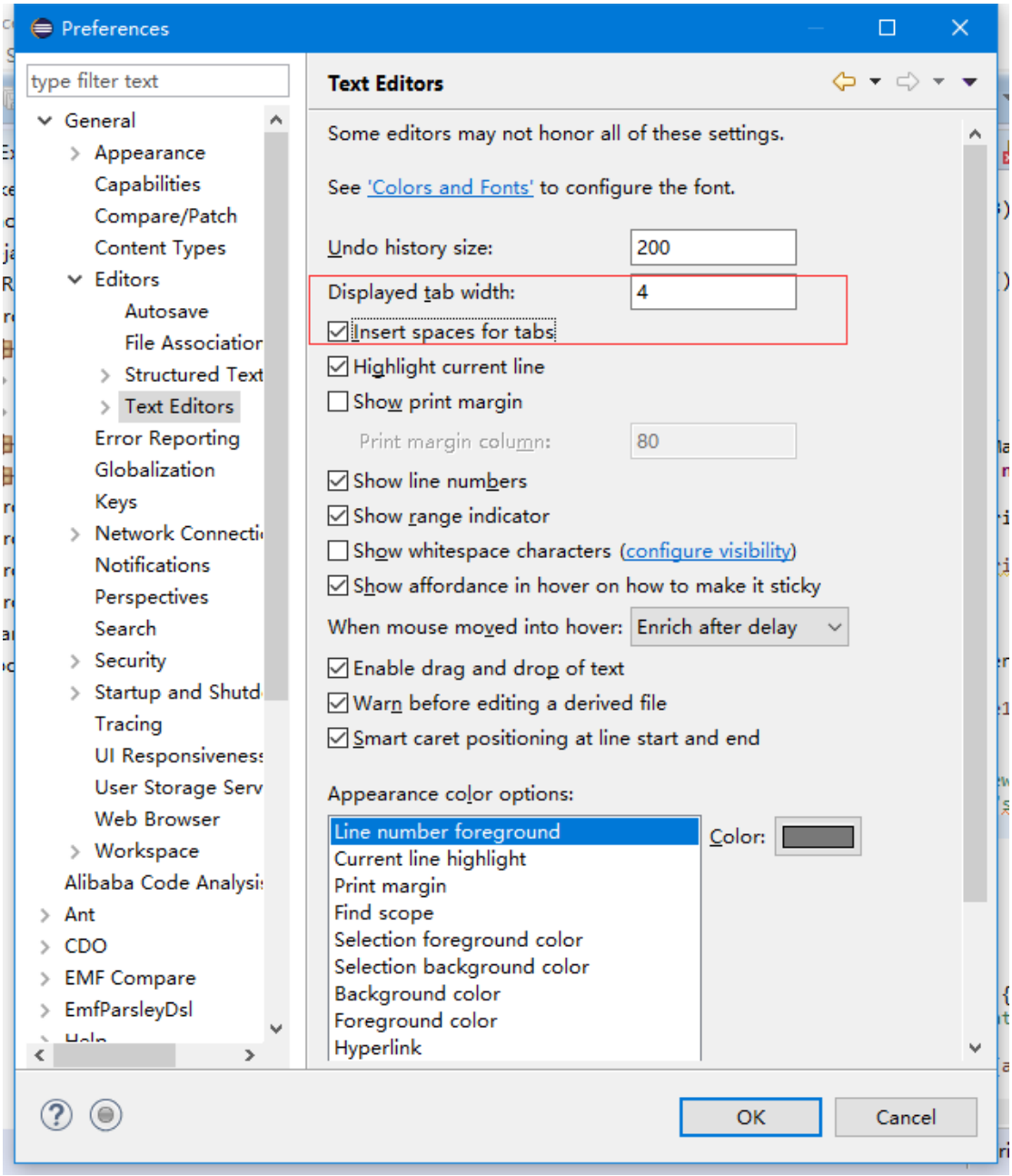
2 编码格式

Eclipse 中首选项->General->Workspace 中”Text file encoding”设置为”UTF-8”，”New text file line delimilter”设置为”Unix”。



3 缩进设置

IDE 中采用 **4 个空格缩进**，禁止使用 tab 字符。如果使用 tab 缩进，必须设置 1 个 tab 为 4 个空格。IDEA 设置时，请勿勾选 Use tab character，而在 eclipse 中，必须勾选 insert spaces for tabs。



六、 相关规定

采用插件方式进行代码规范扫描检查的方法具体可参考《恒生 java 代码规范化扫描方法》。

七、 修订记录

批准日期	修订说明	修订人
2019/01/31	新增	EPG 代码质量组
2019/08/08	调整两项控制级别：修改章节 3.**代码格式**中第 6）、7)两项约定从【强制】级别更新为【推	EPG 代码质量组

	荐】级别	
2019/10/11	修改章节 7 中第 8）项约定从【推荐】级别更新为【参考】级别 修改章节 8 中第 4）项约定从【强制】级别更新为【推荐】级别 修改章节 1 中第 6）项约定从【强制】级别更新为【推荐】级别	EPG 代码质量组
2020/04/07	增加第 4 章第 4 节第 22）、23）两项约定 修改第 4 章第 6 节第 15）项等级为【推荐】级别，并修改用用插件扫描方式进行检查 增加第 4 章第 6 节第 16）项 增加第 4 章第 7 节第 12）项 增加第 4 章第 12 节第 9）项	EPG 代码质量组
2021/4/27	增加第 4 章第 3 节第 13），14）2 项约定 增加第 4 章第 4 节第 24）、25）、26）、27）、28）、29）、30）7 项约定 增加第 4 章第 7 节第 13）、14）2 项约定 增加第 4 章第 8 节第 12）项 修订第 4 章第 5 节第 4）项规约，该项规约调整为“使用集合转数组的方法，必须使用集合的 toArray(T[] array)” 修改第 4 章第 3 节第 9）项，第 4 章第 4 节第 4）、30）项，第 4 章第 7 节第 7）项，第 4 章第 10 节第 4)项,第 4 章第 11 节第 1)项,7）的检查方式为“插件扫描” 删除第 4 章第 6 节第 16）项（与第 4 章第 6 节第 9）项重复）	EPG 研发组
2021/07/08	增加第 4 章第 4 节第 31）项	EPG 研发组
2021/10/26	1、删除 4.4.29 【强制】禁止 import 语句使用*符号 2、删除 4.8.12【强制】变量必须要有 Javadoc 注释，便于代码理解和维护 3、修改 4.7.6 描述， 只规范循环体内做 try catch 操作，定义对象和变量不做限制 4、删除 4.7.7 【强制】循环体中的语句要考量性能，以下操作尽量移至循环体外处理，如获取数据库连接， 进行不必要的 try - catch 操作 （ 这个 try - catch 是否可以移至循环体外 ） 。 5、修改 4.4.26 描述，规范调整为仅对方法内对象型入参做要求	EPG 研发组
2021/11/15	根据研发团队反馈意见，评审修订，详细修订明细如下（所有改动内容都已审阅模式修订）： 1、删除 4.4.29 【强制】禁止 import 语句使用*符号； 2、删除 4.8.12【强制】变量必须要有 Javadoc 注释，便于代码理解和维护； 3、修改 4.7.6 描述， 只规范循环体内做 try catch 操作，定义对象和变量不做限制；	EPG 研发组

	4、删除 4.7.7 【强制】循环体中的语句要考量性能，以下操作尽量移至循环体外处理，如获取数据库连接， 进行不必要的 try - catch 操作（ 这个 try - catch 是否可以移至循环体外 ）； 5、修改 4.4.26 描述，规范调整为仅对方法内对象型入参做要求。	
2023/3/15	1、 4.1.2 检查方式由“插件扫描”改成“人工检查”，当前编码阶段、集成阶段扫描工具没有实现该规则，并且也难以实现用工具检查 2、 4.1.3 参考 p3c， 增加不用遵守 UpperCamelCase 情形 DO/VO/DAO/BO/DAOImpl/YunOS/AP/PO 3、 4.1.16 规范拆分，原有规范描述与约束等级不符，有歧义，拆分成两条 4.1.16 【推荐】代码中浮点数建议使用 BigDecimal，谨慎使用 float、double，根据业务需要定义好精度（计算、入表）。float 和 double 只能用来做科学计算或者是工程计算，在商业计算中我们要用 java.math.BigDecimal。 4.1.17【强制】BigDecimal 禁止使用 double 构造函数 检查方式：插件检查 4、4.1.18 补充规范说明：防止工具类被实例化 5、2.2.1 增加限定条件“条件判断语句里”，仅对条件判断语句里的魔法值做扫描，修改反例。现有插件扫描工具也是仅扫描 if while for 语句里的魔法值 6、4.3.2， 4.3.3， 4.3.4 规范等级改成推荐，检查方式改成人工检查。 历史代码中也大量存在和这个不一致的地方。如果改，只要自动格式化一下，但是改了以后，会导致以后比对代码非常不方便，整篇都是不一致，并且该规则不产生实际价值。 7、4.3.6 检查方式改成 人工检查。 8、4.3.16 规范描述修订为“一行内不能有多条语句”，原有规范描述有歧义 9、增加规范 4.4.8【强制】String 类型之间值的比较，必须使用 equals 方法比较，禁止使用== 或!=进行比较。 检查方式：插件扫描 在 Java 中， 当“==”两侧是基本类型的时候，直接比较值， 当左右两侧为引用类型时，实际上	EPG 研发组

	做的是地址比较。故针对引用类型， 如 string，如果是值比较，应该用 equals 方法， 而不是==（说明： string 类型， 99%的场景都是值比较）。
	10、 4.4.12 规范描述修订，补充构造方法里面只能用来做注入的说明
	11、4.4.26 规范描述修订 导入改成 import， 防止歧义
	12、增加规范 4.4.32【推荐】尽量少用 @Resource 来注入 Bean。
	13、 4.7.6 改为人工检查， 插件无法识别是否是正常业务
	14、 4.8.7， 4.8.8 改为人工检查，参见无法识别
	15、4.9.1 补充 RuntimeException 的说明：Java 原生运行时异常
	16、4.9.2 改成人工检查， 插件无法识别是否是流程/条件控制
	17、4. 9.13 改成人工检查
	18、4.10.1 规范等级改成推荐，检查方式改成人工检查， 规范描述修订成“应用中不建议直接使用日志系统（Log4j，Logback）中的 API，建议使用门面模式的日志框架（如 SLF4J），有利于维护和各个类的日志处理方式统一”
	19、4.10.6 正例修订为“ logger.error("修改证券信息失败，证券信息={}", stockInfo, e); //需要确保 stockInfo.toString()方法已经实现，不建议显式调用 toString 方法，避免日志不输出时（一般 Info 级别以上日志可能会不输出），还会调用 toString 方法，占用业务处理时间。
	”， 当前正例不符合公司发布的日志规范 1）记录日志不推荐字符串拼接。建议使用占位符。 2）禁止打印重复的日志 要求
	20、4.10.8 描述修订， 去除“避免用户投诉时，无所适从”
	21、 4.12.1 检查方式改成人工检查
	22、4.12.9 说明中的非嵌套泛型，当前非嵌套泛型具体明细框架组还未明确
	23、增加规范 4.12.10
	【推荐】服务参数（JresCloudService）应尽量使用单个自定义的 JavaBean，将实际传参封装在 JavaBean 内（在必须使用多个的情况下，应加上注解@CloudFunctionParam）
	24、增加强制级别的插件检查规范 4.12.11
	【强制】日期格式转换禁止使用大写 YYYY（即 new SimpleDateFormat(pattern)中 pattern 不能用 YYYY）
	25、增加推荐级别的插件检查规范 4.12.12
	【推荐】基于 JresCloud 框架开发的产品，业务代码禁用 System.setProperty 方法使用，而应采用 ConfigUtils 提供的对应方法替代使用。	

	26、5.1.1 IntelljIDEA 文件头注释路径修订为“File > Settings > Editor > Copyright > Copyright Profiles”， 与官方推荐的方式保持一致	